

Hunt_Chain

Project 3 — Vulnerability Testing Toolkit

Complete V1 Development & Implementation Report

Project: Hunt_Chain

Component: Project 3 — Vulnerability Testing Toolkit

Version: V1

Status: Completed, Tested, Documented & Frozen

Development Environment: Windows + Python 3.13 + Virtual Environment

Repository: Hunt_Chain

Final Git Commit: d3537b6 — Complete Project 3 Vulnerability Testing Toolkit V1

1. Executive Summary

Project 3 is the **Vulnerability Testing Toolkit** within the Hunt_Chain security-testing platform.

The purpose of Project 3 is to take an authorized attack surface produced by the reconnaissance stage and perform controlled, deterministic vulnerability checks against the discovered targets and endpoints.

The project was deliberately designed as a separate stage in the Hunt_Chain pipeline:

```
Project 1
ScopeGuard
Authorization
    ↓
Project 2
Recon & Attack Surface
    ↓
Project 3
Vulnerability Testing Toolkit
    ↓
Project 4
Manual Testing Companion
    ↓
Project 5
Findings & Reporting Manager
```

Project 3 V1 was developed with a strict scope boundary. The objective was not to build a full exploitation framework or an unrestricted vulnerability scanner. Instead, V1 establishes a safe, modular foundation for controlled vulnerability testing.

The completed implementation contains:

- target and endpoint models
- Project 2 attack-surface input handling
- authorization enforcement
- execution policy
- controlled HTTP/HTTPS transport
- request budgeting
- dry-run support
- vulnerability tester abstraction
- tester registry
- ten frozen V1 vulnerability tests
- evidence collection
- result storage
- result deduplication
- output serialization
- V1 output schema
- output validation
- human-readable output
- controlled localhost vulnerable lab
- unit tests
- integration tests
- complete regression testing
- project documentation

The final implementation was committed to Git and pushed to the Hunt_Chain GitHub repository.

Project 3 V1 is now considered **frozen**.

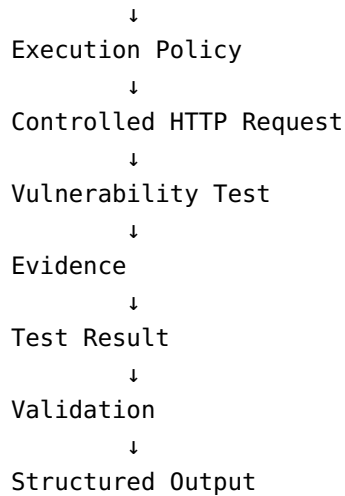
2. Project Objective

The primary objective was to create the third component of Hunt_Chain responsible for controlled vulnerability testing.

Project 3 receives reconnaissance information from Project 2 and transforms it into a controlled testing workflow.

The intended execution path is:

```
Reconnaissance Input
    ↓
Target Construction
    ↓
Authorization
```



The project was designed around four major principles:

1. **Authorization before execution**
2. **Controlled and bounded network activity**
3. **Deterministic and explainable vulnerability detection**
4. **Strict V1 scope discipline**

3. Hunt_Chain Project Architecture

Hunt_Chain is divided into five separate projects.

Project	Responsibility
Project 1 — ScopeGuard	Authorization and scope validation
Project 2 — Recon & Attack Surface Suite	Reconnaissance
Project 3 — Vulnerability Testing Toolkit	Vulnerability testing
Project 4 — Manual Testing Companion	Deeper manual testing
Project 5 — Findings & Reporting Manager	Findings and reporting

Project 3 therefore does not attempt to perform the responsibilities of Projects 1, 2, 4, or 5.

This separation prevents the vulnerability-testing layer from becoming an uncontrolled combination of reconnaissance, exploitation, and reporting functionality.

4. V1 Scope Discipline

A major development rule for Project 3 was:

```
V1 requirement missing
    → implement/fix

V1 bug
    → fix

Useful but non-V1 feature
    → do not implement
```

Future functionality is treated as:

```
DEFERRED – V2+
```

or:

```
NOT A V1 REQUIREMENT
```

This prevented scope expansion during implementation.

The ten vulnerability tests were frozen before implementation and were not expanded during development.

5. Project 3 V1 Architecture

The final source tree is organized into functional layers:

```
src/
├─ hunt_chain_vulnerability/
│   ├── authorization/
│   ├── config/
│   ├── execution/
│   ├── input/
│   ├── lab/
│   ├── models/
│   ├── policy/
│   └── results/
```

```
├─ testers/  
└─ transport/
```

Each layer has a specific responsibility.

6. Input Layer

The input layer is responsible for converting reconnaissance information into Project 3's internal representation.

It contains:

```
input/  
└─ attack_surface.py
```

The attack-surface models represent:

- assets
- endpoints
- target relationships
- schema information
- reconnaissance metadata

The layer supports the handoff from Project 2 into Project 3.

Project 3 does not perform unrestricted target discovery.

7. Target and Endpoint Models

The project contains structured target and endpoint models.

A target contains information such as:

- target ID
- asset ID
- host
- scheme
- port
- metadata

An endpoint contains:

- endpoint ID
- target ID
- URL
- HTTP method
- path
- parameters
- metadata

Validation is performed when these models are created.

This gives the testing layer predictable structured input instead of arbitrary strings.

8. Authorization Architecture

Authorization is one of the most important security controls in Project 3.

The execution sequence is:

```
Target
↓
Authorization Provider
↓
Authorization Result
↓
Authorization Policy
↓
Execution Controller
```

The supported authorization states are:

```
IN_SCOPE
OUT_OF_SCOPE
UNKNOWN
CONFLICT
```

Only:

```
IN_SCOPE
```

is permitted to proceed.

The other states are blocked.

```
IN_SCOPE
  ↓
EXECUTE

OUT_OF_SCOPE
  ↓
BLOCK

UNKNOWN
  ↓
BLOCK

CONFLICT
  ↓
BLOCK
```

This is a fail-closed design.

An unknown authorization decision does not become permission.

9. Static Authorization Provider

A deterministic static authorization provider was implemented for controlled testing.

It maps target identifiers to predefined authorization decisions.

If a target has no authorization decision, the provider returns:

```
UNKNOWN
```

This behavior is intentional.

It ensures that missing authorization information cannot accidentally permit testing.

10. Execution Policy

Project 3 includes an explicit execution policy.

The execution configuration contains:

```
timeout
rate_limit
concurrency
max_requests_per_test
max_redirects
verify_tls
```

The V1 execution model uses:

```
concurrency = 1
```

This provides predictable and bounded execution.

The project does not implement aggressive parallel scanning in V1.

11. Request Budget

A request budget was implemented to limit the number of network requests that a vulnerability test may make.

The purpose is to prevent an individual tester from generating unrestricted traffic.

The request budget works together with the execution policy.

This establishes a controlled testing boundary rather than allowing each vulnerability test to independently generate unlimited requests.

12. Dry-Run Mode

Project 3 supports dry-run execution.

Dry-run behavior is designed to allow execution logic to be validated without sending network traffic.

In dry-run mode:

```
Authorization
↓
```



```
Validation
  ↓
Request preparation
  ↓
NO NETWORK REQUEST
```

This provides a safer way to verify execution decisions.

13. Controlled HTTP Transport

The transport layer contains:

```
transport/
├─ base.py
├─ http.py
├─ models.py
└─ __init__.py
```

The main implementation is the controlled HTTP transport.

It supports:

- HTTP
- HTTPS
- explicit request timeout
- TLS verification
- request budgeting
- controlled response capture
- HTTP response metadata

The transport rejects unsupported schemes.

It also rejects URLs containing embedded credentials.

14. HTTP Request and Response Models

The project uses structured request and response models.

The response model stores information such as:

- status code

- headers
- response body
- elapsed time
- final URL
- redirect count
- metadata

The response model also provides a SHA-256 body hash.

A bounded body excerpt can be generated for evidence.

This avoids unnecessarily storing complete response bodies as evidence.

15. Execution Controller

The ExecutionController acts as the central execution boundary.

Its responsibilities include:

1. receiving a target
2. requesting authorization
3. enforcing authorization policy
4. enforcing dry-run behavior
5. invoking controlled transport
6. returning the controlled response

The controller prevents unauthorized targets from reaching network execution.

16. Vulnerability Tester Framework

The tester framework defines a common contract for every vulnerability test.

Each tester provides:

```
definition
can_test(context)
execute(context)
```

This means that individual vulnerability tests remain modular.

The architecture allows the tester registry to discover and execute the approved V1 test implementations without coupling the testers directly to the entire application.

17. Test Definitions

Each test has a structured definition containing:

- test ID
- name
- description
- vulnerability class
- version
- prerequisites
- allowed methods
- request strategy
- detection logic
- evidence requirements
- safety constraints
- metadata

This separates the definition of a test from the execution result.

18. V1 Vulnerability Test Set

The final V1 test set contains exactly ten tests.

V3-001 — Security Header Analysis

Checks recommended HTTP security headers.

The V1 implementation evaluates headers including:

- Content-Security-Policy
- Strict-Transport-Security
- X-Content-Type-Options
- Referrer-Policy
- Permissions-Policy

Missing security headers can produce a `FAIL` result with high confidence.

V3-002 — Cookie Security Attribute Check

Analyzes `Set-Cookie` responses.

The test checks security attributes including:

- Secure
- HttpOnly
- SameSite

If cookies are absent, the result may be:

NOT_APPLICABLE

Missing security attributes produce a security observation.

V3-003 — HTTPS / Transport Security Check

Evaluates transport security.

The implementation distinguishes between HTTP and HTTPS.

HTTP transport is treated as an insecure transport observation.

For HTTPS, the test also considers HSTS behavior.

V3-004 — HTTP Method Exposure Check

Analyzes the HTTP `Allow` header.

Unexpected methods beyond the expected safe methods can produce:

POTENTIAL

with manual validation required.

The test is intentionally detection-oriented rather than automatically exploiting an exposed method.

V3-005 — Directory Listing Detection

Checks response bodies for directory-listing indicators such as:

Index of /
Directory listing for
Parent Directory

Detection produces a potential result requiring manual validation.

V3-006 — Information Disclosure Detection

Checks response bodies for common information-disclosure indicators.

Examples include indicators associated with:

- stack traces
- debug output
- traceback information
- diagnostic pages
- environment/debug information

The result is designed as an indicator rather than an automatic confirmation of a vulnerability.

V3-007 — Sensitive Resource Exposure Check

Checks paths and successful responses for indicators associated with sensitive resources.

Examples include:

```
.env  
.git  
config.php  
wp-config.php  
credentials  
password  
secret  
DB_PASSWORD
```

The test is bounded and detection-oriented.

Potential exposure requires manual validation.

V3-008 — Reflected Input Indicator Check

The test uses a controlled input marker.

If the exact marker is reflected in the response body, the test produces a potential reflection result.

This does not automatically classify the response as a confirmed injection vulnerability.

Manual validation is required.

V3-009 — Basic Authentication/Authorization Response Consistency Check

This test compares controlled responses.

It requires comparison responses and checks whether meaningful differences exist between authentication/authorization states.

Differences in status or body hash can produce a potential result.

The test is intentionally basic and does not implement credential attacks.

V3-010 — Error Information Disclosure Check

Analyzes error responses for information-disclosure indicators.

The test focuses on error responses such as:

4xx
5xx

and searches their bodies for diagnostic information.

Tracebacks and similar details can result in a potential disclosure observation.

19. Result Status Model

Project 3 supports six result statuses:

```
PASS
FAIL
POTENTIAL
NOT_APPLICABLE
SKIPPED
ERROR
```

This allows the system to distinguish:

- successful security checks
- detected failures
- indicators requiring manual validation
- tests that do not apply
- intentionally skipped tests
- execution errors

20. Confidence Model

Confidence is qualitative rather than an unexplained numerical score.

The supported levels are:

```
LOW
MEDIUM
HIGH
```

`FAIL` and `POTENTIAL` results require confidence.

`POTENTIAL` results also require:

```
manual_validation_required = true
```

This keeps automated detection separate from human confirmation.

21. Evidence Architecture

Project 3 includes a dedicated evidence model.

Evidence may contain:

- evidence ID
- evidence type
- target
- test ID
- observation
- timestamp
- HTTP status code
- response headers
- body hash
- bounded response excerpt
- metadata

Evidence is stored separately from test results.

Results reference evidence through evidence IDs.

22. Evidence Safety

The project avoids unnecessarily storing complete sensitive HTTP responses.

Instead, evidence can contain:

```
HTTP status
headers
body hash
limited body excerpt
metadata
```

The body excerpt is bounded to a maximum of 4096 characters.

This provides useful verification material while limiting unnecessary sensitive-data retention.

23. Result Store

The result layer includes:

```
results/
├─ store.py
├─ builder.py
```



```
|— serializer.py
|— json_export.py
|— schema.py
|— validator.py
└— human.py
```

The ResultStore manages:

- results
- evidence
- result lookup
- evidence lookup
- deterministic ordering
- deduplication

24. Deduplication

Results are deduplicated using:

```
target_id
endpoint_id
test_id
```

This forms the logical deduplication key.

Therefore, the same vulnerability test cannot create multiple logical results for the same target and endpoint within the same result store.

25. Output Builder

The output builder constructs a structured Project 3 V1 output.

The output includes information such as:

- schema version
- run information
- authorization
- target
- tests
- results
- evidence

- errors
- metadata

The output is versioned as:

v1

26. Output Schema

Project 3 contains a dedicated V1 output schema implementation.

The schema is identified using the Project 3 V1 schema namespace.

This ensures that output can be validated structurally and versioned independently from future output formats.

The schema is implemented inside the results layer rather than creating an unrelated duplicate schema implementation.

27. Output Validation

The output validator checks conditions including:

- correct schema version
- unique result IDs
- unique evidence IDs
- valid test IDs
- valid evidence references
- confidence requirements
- manual validation requirements

This prevents malformed output from being exported as valid V1 output.

28. Human-Readable Output

Project 3 provides a basic human-readable summary.

The purpose is to make V1 results understandable without turning Project 3 into the reporting system.

Professional findings management and reporting are intentionally reserved for Project 5.

29. Controlled Local Vulnerable Lab

A dedicated localhost vulnerable application was implemented:

```
src/hunt_chain_vulnerability/lab/vulnerable_app.py
```

The lab runs on:

```
127.0.0.1:8765
```

It provides deterministic vulnerable responses for Project 3 testing.

30. Local Lab Routes

The lab includes routes for the V1 tests.

Route	Purpose
/	Baseline response and cookie
/reflect	Reflected input
/directory	Directory listing
/error	Error disclosure
/sensitive	Sensitive resource exposure
/auth-required	Authentication behavior
OPTIONS /	HTTP method exposure

The application is intentionally vulnerable.

It is designed only for controlled local testing.

31. Local Lab Lifecycle

The integration test pipeline can start and stop the lab automatically.

The user does not need to manually run the HTTP server for the automated integration test.

This reduces setup complexity and prevents unnecessary persistent lab services.

32. Testing Strategy

Project 3 uses layered testing:

```
graph TD; A[Unit Tests] --> B[Component Tests]; B --> C[Pipeline Tests]; C --> D[Integration Tests]; D --> E[Authorized Local Lab];
```

Unit Tests
↓
Component Tests
↓
Pipeline Tests
↓
Integration Tests
↓
Authorized Local Lab

This gives the project several levels of verification.

33. Unit Test Coverage

Unit tests cover the major components.

The test suite includes coverage for:

- attack-surface models
- authorization
- authorization policy
- configuration loader
- evidence
- execution controller
- execution policy
- HTTP transport
- local lab
- general models
- output builder

- results
- result validation
- tester contract
- tester registry
- test definitions
- transport models

34. Integration Testing

Integration tests verify the interaction between:

```
Authorization
  ↓
Execution
  ↓
HTTP Transport
  ↓
Vulnerability Testing
  ↓
Local Lab
```

The local lab provides deterministic responses so that vulnerability detection can be tested without relying on external websites.

35. Final Regression Test

The final verification command was:

```
pytest -q -W error
```

The complete suite reached:

```
100%
```

with no reported test errors.

Warnings were also treated as errors during the final verification.

This was important because earlier development produced Pytest collection warnings caused by class names beginning with `Test` .

Those warnings were corrected rather than ignored.

36. Development Batches

Project 3 was developed through eight major batches.

Batch 1 — Foundation & Data Models

Implemented:

- project configuration
- package structure
- enums
- target models
- authorization models
- test definitions
- evidence models
- result models
- tester base contract
- initial tests

Initial verification reached:

19 passed

The Pytest collection warnings were subsequently corrected.

Batch 2 — Project 2 Input & Authorization Boundary

Implemented:

- attack-surface input models
- Project 2 conversion logic
- authorization provider
- authorization policy
- authorization enforcement

Verification reached:

33 passed

and warning-as-error verification also passed.

Batch 3 — Execution Policy & Controlled HTTP Transport

Implemented:

- execution policy
- request budget
- HTTP request model
- HTTP response model
- controlled HTTP transport
- dry-run network protection
- TLS verification controls

Verification reached:

54 passed

Batch 4 — Configuration & Execution Controller

Implemented:

- configuration loader
- execution controller
- authorization-to-execution boundary
- configuration integration

During this stage, integration mismatches were corrected, including import/export issues.

Final verification reached:

69 passed

Batch 5 — Tester Framework & Registry

Implemented:

- tester registry
- frozen V1 test catalog
- tester registration
- duplicate test-ID prevention

Pytest collection warnings caused by imported `Test...` names were fixed.

Verification reached:

84 passed

with warning-as-error testing passing.

Batch 6 — Ten V1 Vulnerability Tests

Implemented all ten frozen tests:

V3-001
V3-002
V3-003
V3-004
V3-005
V3-006
V3-007
V3-008
V3-009
V3-010

The tests use controlled response analysis and structured evidence.

Batch 7 — Results, Evidence, Deduplication & Output

Implemented:

- ResultStore
- evidence storage
- result builder

- serializer
- JSON exporter
- human-readable output
- V1 output schema
- result validation
- deduplication

This completed the result pipeline.

Batch 8 — Local Lab, Integration, Documentation & Freeze

Implemented and verified:

- local vulnerable lab
 - integration pipeline
 - lab tests
 - documentation
 - cleanup of stale tester code
 - cleanup of duplicate lab implementation
 - final regression testing
 - Git commit
 - GitHub push
 - V1 freeze
-

37. Cleanup Performed Before Freeze

Two stale/duplicate components were identified during the final audit.

Stale tester registry

An obsolete:

```
testers/v1.py
```

existed from an earlier architecture attempt.

It was not referenced by the final architecture and used incompatible tester constructors.

It was removed.

Duplicate lab

An older:

```
tests/lab/vulnerable_app.py
```

duplicated the authoritative source lab.

The source-package lab was retained as the single authoritative implementation.

38. Git Safety

A particularly important repository concern was that Project 1 already contained unrelated working-tree changes.

The Project 3 commit was therefore staged explicitly rather than using a broad:

```
git add .
```

The staged commit contained Project 3 only.

The unrelated Project 1 changes remained outside the Project 3 commit.

This prevented accidental modification or publication of unrelated Project 1 work.

39. Git Commit

The main Project 3 V1 commit was:

```
d3537b6  
Complete Project 3 Vulnerability Testing Toolkit V1
```

The commit contained the Project 3 implementation, configuration, tests, documentation present at that checkpoint, and project files.

A subsequent documentation commit added the remaining installation documentation.

40. GitHub Publication

The Project 3 work was successfully pushed to the Hunt_Chain repository.

The remote reported that the repository had moved to:

```
https://github.com/Atharva-g19/Hunt_Chain.git
```

The push completed successfully.

The branch was synchronized with:

```
origin/main
```

41. Final Project Structure

The important final structure is:

```
Project_3_Vulnerability/
|
├── .gitignore
├── LICENSE
├── README.md
├── pyproject.toml
|
├── config/
|   ├── default.yaml
|   └── lab.yaml
|
├── docs/
|   ├── architecture.md
|   ├── authorization.md
|   ├── configuration.md
|   ├── installation.md
|   ├── local_lab.md
|   ├── project2_input.md
|   ├── results_and_evidence.md
|   ├── testing.md
|   ├── usage.md
|   ├── v1_freeze.md
|   └── v1_scope.md
```

```
|
|— schemas/
|
|— src/
|   └─ hunt_chain_vulnerability/
|       └─ authorization/
|       └─ config/
|       └─ execution/
|       └─ input/
|       └─ lab/
|       └─ models/
|       └─ policy/
|       └─ results/
|       └─ testers/
|       └─ transport/
|
|— tests/
|   └─ integration/
|   └─ unit/
```

42. Security Controls Implemented

The major V1 safety controls are:

Authorization gate

Only authorized targets execute.

Fail-closed authorization

Unknown or conflicting authorization is blocked.

Dry-run

Network requests can be prevented entirely.

Request timeout

HTTP requests have an explicit timeout.

Request budget

Tests have a maximum request allowance.

Concurrency control

V1 uses single-lane execution.

Rate control

Request-rate configuration is available.

TLS verification

HTTPS certificate verification is configurable.

Embedded credential rejection

URLs containing embedded credentials are rejected by the controlled transport.

Bounded evidence

Response excerpts are limited.

Deterministic detection

Test logic is explicit and explainable.

Manual validation

Potential results require human validation.

43. Explicitly Outside Project 3 V1

The following functionality was intentionally not implemented:

- full exploitation framework
- credential stuffing
- password spraying
- brute-force authentication
- persistence
- lateral movement
- malware
- ransomware
- post-exploitation
- privilege-escalation framework
- WAF bypass automation
- stealth/evasion
- botnet functionality

- mass internet scanning
- unrestricted target discovery

These capabilities remain outside the frozen V1 scope.

44. Project Responsibility Boundaries

Project 3 does not replace:

Project 1

ScopeGuard remains responsible for authorization and scope validation.

Project 2

Recon remains responsible for attack-surface discovery and observation.

Project 4

Manual Testing Companion will handle deeper manual security testing.

Project 5

Findings & Reporting Manager will handle full findings management and reporting.

This separation is a fundamental part of the Hunt_Chain architecture.

45. V1 Definition of Done

The project has completed the intended V1 milestones:

Requirement	Status
Architecture	Complete
Configuration	Complete
Project 2 input	Complete
Authorization integration	Complete
Execution policy	Complete
Controlled HTTP transport	Complete

Requirement	Status
Tester framework	Complete
Ten approved tests	Complete
Evidence model	Complete
Result model	Complete
Deduplication	Complete
Output schema	Complete
Human-readable output	Complete
Validation	Complete
Unit tests	Complete
Integration tests	Complete
Local lab	Complete
Documentation	Complete
Git commit	Complete
GitHub publication	Complete
V1 scope freeze	Complete

46. Final V1 Status

Project 3 has reached the following state:

HUNT_CHAIN PROJECT 3 – V1	
Architecture	COMPLETE
Authorization	COMPLETE
Execution Controls	COMPLETE
HTTP Transport	COMPLETE
Tester Framework	COMPLETE
10 V1 Tests	COMPLETE
Evidence	COMPLETE
Results	COMPLETE
Output Schema	COMPLETE
Validation	COMPLETE
Local Lab	COMPLETE

Integration	COMPLETE
Regression Testing	COMPLETE
Documentation	COMPLETE
GitHub	COMPLETE
V1 STATUS	FROZEN 

47. Conclusion

Project 3 successfully establishes the vulnerability-testing layer of Hunt_Chain.

The completed V1 provides a controlled pipeline from reconnaissance input through authorization, bounded HTTP execution, vulnerability testing, evidence collection, result validation, and structured output.

The project deliberately avoids becoming an unrestricted offensive-security framework. Its architecture emphasizes authorization, safety controls, deterministic detection, evidence integrity, modular testers, and clear separation of responsibilities between Hunt_Chain projects.

The ten approved vulnerability tests provide the initial V1 testing capability, while the modular tester framework allows future versions to evolve without changing the fundamental execution architecture.

Project 3 has been:

```
Implemented
  ↓
Tested
  ↓
Integrated
  ↓
Documented
  ↓
Committed
  ↓
Pushed to GitHub
  ↓
FROZEN
```

Project 3 — Vulnerability Testing Toolkit V1 is complete.